GPT4o-mini-based Mutant Selection for Java Mutation Testing

Pham Manh Quynh, Tran Truc Vy, Le Nguyen Dinh Khoi, Nguyen Truong Vinh, Nguyen Nhat Anh
FPT University, HCM, VietNam

***Abstract*—Mutation testing reliably evaluates test suite quality, but its execution cost is very high, making the selection of a useful mutant subset an important practical problem. Prior studies have used LLMs to generate mutants or detect equivalent mutants, but none have evaluated an LLM as a mutant-selection decision maker against random selection under the same budget. We use GPT-4o-mini (temperature=0, fixed prompt) to rank 11,040 PIT mutants from three Defects4J projects (Commons Codec, Commons CSV, JacksonCore; versions 1f–10f), select the top 10% within each (project, version, class) group, and compare it with random selection under the same budget (seed 42) using mutation score. GPT-4o-mini achieves an overall mutation score of 0.716 compared to 0.670 for random selection (+4.6 points); a one-sided Wilcoxon test at the group level gives p=0.059, r_rb=+0.350 (not meeting the full pre-registered criteria), while mutant-weighted analysis (p=0.0097) and a 5,000-draw Monte Carlo simulation (p=0.0002) support GPT's advantage. These results indicate that LLM-based mutant selection provides concentrated benefits on large, complex parser classes but does not consistently outperform random selection across all groups — consistent with earlier findings on naturalness-based selection.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Mutation testing is a technique used to evaluate a test suite by creating mutants and testing whether the test suite can recognize them. However, mutation testing often creates an enormous number of mutants. This huge number of mutants significantly increases the practical cost and makes mutation testing hard to apply in practice. This financial cost is one reason for a long line of research related to mutant reduction/selection that has existed for years [1], [2], [3]. Choosing a small mutant subset that is "worth running" would provide practical value for developers, CI pipelines, or even at industrial scale.

Recent studies have brought LLMs into mutation testing in two main ways. The first way is generating semantic mutants: Wang et al.'s comprehensive research on 851 real bugs in Java (Defects4J 2.0 + ConDefects) shows that mutants generated from LLMs reach 76.47% real-bug detection compared to 44.15% for rule-based techniques - a 1.75x improvement - even though this comes at the price of a non-compilable rate, duplicate rate, and higher equivalent rate of 32.73 / 9.38 / 3.64 percentage points, respectively [4]; after that, SMART raised validity from 42.89% to 65.6% and real-bug detection to 92.61% due to RAG + focused chunking + fine-tuning [5]; LLMorpheus generates mutants through prompt templates [6]. The second way is detecting/filtering equivalent mutants with 98% precision [7]; Tian et al. evaluate the LLM system's ability to detect equivalent mutants [8]. Besides that, predictive mutation testing (PMT) trains models to predict the kill/survive result to reduce practical costs: Seshat predicts the kill matrix via a natural language channel in code [9], MutationBERT improves prediction using CodeBERT [10], and SODA surpasses these baselines with kill-F1 improvements ranging from 5.32% to 114.92% depending on configuration [11]. Finally, selective mutation research learns how to build or select valuable mutants: LEAM/LEAM++ learn how to create filtering mutants [12], [13], Cerebro chooses subsuming static mutants [14], and MuDelta chooses mutants related to a commit [15]. Overall, machine learning has proven its potential to reduce mutation testing costs across multiple stages.

However, no study has evaluated GPT-4o-mini as a mutant selection decision maker or directly compared it against random fixed-seed selection under an identical mutant budget. The common thread in the above LLM studies is that they use LLMs to create mutants, filter equivalent mutants, and generate tests. There is no research that uses an LLM as a mutant selector to choose a subset and compares it with random selection under the same mutant budget and the same metrics. The closest research that matches this design is Jimenez et al. [16], who compared naturalness-based selection (n-gram, not LLM) with random selection on 5 Defects4J projects and concluded that naturalness-based selection is only equivalent to (slightly inferior to) random selection, with a small effect size - this leaves the question of whether a modern LLM can perform better than an n-gram language model in the same role of selecting mutants. PRIMG [17] uses LLM prioritization but on Solidity smart contracts and for test generation, and it still does not address the choose-vs-random problem in Java.

This paper fills this gap through a comparative experiment, pre-registered with a full hypothesis, seed, and criteria before running. We use GPT-4o-mini to rate the usefulness, from 1 to 5, of all 11,040 PIT mutants from three Defects4J projects belonging to three different domains - Commons Codec (encoding), Commons CSV (CSV parsing), and JacksonCore (JSON parsing) - then choose the top 10% from each (project, version, class) group and compare it with random selection under the same budget and on the same pool. Unlike other studies that generate mutants and have to process non-compilable mutants, our design keeps PIT's valid mutant pool intact, so that all differences in mutation scores can be attributed *entirely* to the quality of the selection decision. Besides the main numbers, we also analyze the two ways of combining valid results (by group and by mutant), leading to two different statistical conclusions on the same data - a phenomenon that, to our knowledge, has

not been reported in any article on mutant selection.

To summarize, this paper contributes:

1) the first empirical evaluation of GPT-4o-mini as a mutant-selection decision-maker against same-budget random selection, using mutation score as the primary metric on 11,040 PIT mutants from three Defects4J projects spanning three domains (encoding, CSV parsing, JSON parsing).
2) Experimental results showing a +4.6-point overall mutation-score advantage that is not uniform across groups (group-weighted p=0.059 versus mutant-weighted Monte-Carlo p=0.0002) - a unit-of-analysis finding that matters for future selection studies.
3) A publicly available replication package - dataset, prompts, pipeline, and raw results.

The rest of this paper is structured as follows: Section 2 reviews related work according to three themes. Section 3 describes the methodology, including the Defects4J dataset and mutant generation using PIT, the mutant selection pipeline for GPT-4o-mini and random fixed-seed selection under the same mutant budget, the mutation score metric, and the statistical tests and thresholds that were pre-registered. Section 4 presents the experimental results and reports the results for each research question. Section 5 discusses the reasons for and meaning of the results, especially the differences between the projects and the method's applicability in practice. Section 6 analyzes threats to validity and the limitations of the experimental design. Section 7 concludes and suggests directions for further research.

II. Related Work

A. LLM in mutation testing (generation and filtering)

The most active branch of research today uses LLMs to generate mutants. Wang et al. performed the first comprehensive study with many models (GPT-4o-mini, GPT-3.5-Turbo, DeepSeek-V3, CodeLlama-13B, StarChat-B...) on 851 real bugs in Java, including 605 bugs from 12 Defects4J 2.0 projects and 246 ConDefects bugs [4]. Key result: mutants generated by LLMs detect 76.47% of real bugs compared to 44.15% for rule-based techniques (Major, LEAM, uBERT) - meaning an improvement of 32.99 percentage points - but the trade-off is that the non-compilable ratio was higher by 32.73 points, the duplicate ratio higher by 9.38 points, and the equivalent ratio higher by 3.64 points. SMART addressed these disadvantages using RAG on vectorized real-bug repositories, focused code chunking, and supervised fine-tuning: validity increased from 42.89% to 65.6%, the non-duplicate rate from 87.38% to 95.63%, and real-bug detection reached 92.62% compared to 57.86% for LLMut [5]. LLMorpheus takes a lighter approach using prompt templates [13].

On the equivalent mutant filtering side - a well-known undecidable problem in mutation testing - Tian et al. evaluated a system of LLMs [8], and GEM-LLM combined inter-procedural slicing, invariant inference via LLM, and SMT verification on 10 Defects4J projects (PIT mutants): classifying 25-30% of surviving mutants that had been overlooked as equivalent, with 98% precision, reaching up to 34.2% for relational operators and 31.8% for conditional-boundary operators, broken down by operator [7].

LLMs are also used to generate mutation-guided tests: MuTAP uses mutants as feedback for the program [18]; MUTGEN (Llama-3.3-70B + PIT feedback) surpasses EvoSuite in mutation score, averaging 88-91% on HumanEval-Java and LeetCode-Java [**?**].

Common limitations of this group: in all the studies above, LLMs play a role in generating, filtering, or generating tests, but none of the papers use an LLM on a pre-existing list of mutants and ask: "with a fixed budget, which mutants should be run?"

B. Predictive mutation testing and selective mutation

PMT avoids execution costs by predicting the result: Seshat uses natural language (method name, test name) to predict the kill matrix faster than running the actual tests [9]; MutationBERT uses CodeBERT with the full context of the mutant-test pair, significantly improving same-project and cross-project prediction [10]; SODA (CodeT5 + contrastive learning) currently leads on six Defects4J v2.0.0 projects, with kill-F1 improvements of 5.32-114.92%, survive-F1 improvements of 0.04-25.54%, and accuracy improvements of 4.25-60.43% compared to baselines, and compared to MutationBERT in the cross-project scenario, it achieves +11.09% kill-F1 [11]. On the selective mutation side, Cerebro selects subsuming mutants via static analysis [14]; LEAM and LEAM++ directly learn to build quality mutants instead of selecting from a pool [15]; AL-PMT shows that, with active learning, observing only 10% of the kill status of the pool achieves 98% of maximum performance in ¿80% of projects - independent evidence that a 10% budget is a meaningful working point in practice [19]. Delgado-Pérez et al. summarized 8 methodological threats affecting PMT approaches as a whole - from data leakage to evaluation unit selection - and experimentally demonstrated their impact [20]; our design directly incorporates these warnings (pre-registration, pairing by project/version/class, reporting both pooled units).

Common limitations of this group: these approaches need to be trained or fine-tuned per project/dataset in order to predict the result or build new mutants; they do not answer the question, "Is a training-free, zero-shot LLM subset better than random?" - a question that is meaningful for teams that lack training facilities.

C. Selection compared with a random baseline

Random selection has long been a compulsory baseline for the mutant selection problem: Zhang et al. systematically compared data evaluating mutant selection [21], while Pitts reconsidered random selection under the influence of equivalent mutants [21]. Sentinel approached the problem via hyper-heuristics to generate reduction strategies [1]. Most directly relevant to us, Jimenez et al. compared naturalness-based ranking (n-gram) against random selection on five Defects4J projects (¿100k mutants, 230 real faults) using the same statistical tools we inherited - the Wilcoxon

Signed-rank test and effect size - and obtained negative results: $A_{12} \approx 0.57 - 0.58$ (small), meaning naturalness is not significantly better than random selection in a way that is meaningful in practice [16]. This number is the basis for our setting the effect size threshold $r\_rb \geq 0.30$ (medium) in the proposal: an improvement at the level Jimenez observed would not be considered a success.

D. Positioning

Unlike prior work, this paper is the first to evaluate GPT-4o-mini as a mutant-selection decision-maker against same-budget random fixed-seed selection on Defects4J, inheriting the ranking-versus-random evaluation design of Jimenez et al. [16] but replacing n-gram naturalness with a modern instruction-following LLM, and additionally analyzing how the unit of aggregation (group-weighted versus mutant-weighted) affects the statistical conclusion.

III. Methodology

A. Dataset

Source and reason: Our experiment uses Defects4J [22] (https://github.com/rjust/defects4j) - a public Java benchmark containing real bugs with included test suites, which appears in almost every recent mutation testing study [16], [9], [11], [7], [4], ensuring reproducibility and comparability with prior work.

Projects and range: Three projects across three different domains are used to increase external validity: Commons Codec (encoding/phonetic algorithms: Base64, DoubleMetaphone, Caverphone...), Commons CSV (CSV parsing: CSVParser, CSVPrinter, CSVFormat...), and JacksonCore (JSON parsing: UTF8StreamJsonParser, ReaderBasedJsonParser, TextBuffer...). Each project has 10 buggy versions (1f-10f).

Mutant pool: PIT [23] (pitest 1.15.8, default mutator set: NegateConditionals, Math, ConditionalsBoundary, VoidMethodCall, ReturnVals...) generates mutants on the classes affected by Defects4J. Final result: Codec 2,317, CSV 603, JacksonCore 8,120 - a total of 11,040 mutants across 20 classes, creating 37 (project, version, class) groups used as the pairing unit.

Ground truth: The status of each mutant (KILLED / SURVIVED / NO_COVERAGE / TIMED_OUT / NON_VIABLE) is determined by PIT when running the developer's original test suite - completely automated, with no manual labeling step, so inter-annotator agreement does not apply. Descriptive statistics: the overall pool KILLED rate is 84.9% for Codec (1,966/2,317), 63.2% for CSV (381/603), and 62.2% for JacksonCore (5,047/8,120); the NO_COVERAGE rates are 1.3%, 6.6%, and 24.4%, respectively.

Data issue detected and fixed (transparently recorded): On the first PIT run, Codec returned 100% NO_COVERAGE (0/2,317 killed), uniformly across all 10 versions. Diagnosis showed that Defects4J's tests ran cleanly (Failing tests: 0), but PIT reported "could not run any tests ... a recent version of JUnit 4 must be on the classpath": Codec's old test version only included JUnit 3.8.x, while PIT requires JUnit 4 to run them through JUnit38ClassRunner. After adding junit 4.13.2 + hamcrest-core 1.3 at the beginning of the classpath and rerunning via the PIT command-line, Codec produced a normal result (84.9% killed), independently verified on version 1f before running all 10. CSV's and JacksonCore's ground truths were confirmed to be constant byte-for-byte; the broken results are saved as evidence. Following this issue, an auto-guard was added to the pipeline: it fails if any project has 0 mutants KILLED or $> 50\%$ NO_COVERAGE. Details are in amendment_v1.2.md.

B. Experimental Setup and Pipeline

Model: We use GPT-4o-mini through the OpenAI Chat Completions API; the actual model version returned by the API is GPT-4o-mini-2024-07-18 [24]. Configuration: temperature = 0, batch of 8 mutants/request, maximum of 4 retries with exponential backoff; checkpoints are recorded after each successful batch to withstand network outages/session terminations (resuming does not lose data and does not re-call already-processed mutants).

Prompt: The ranking prompt is kept fixed for the entire experiment (quoted verbatim - in LaTeX it is included as:

```
You are ranking Java PIT mutants for mutation
    testing.

For each mutant, assign:
- score: integer from 1 to 5
  5 = highly valuable mutant, likely
      semantically meaningful and useful for
      testing
  4 = useful mutant
  3 = moderate mutant
  2 = weak mutant
  1 = low-value or trivial mutant
- confidence: integer from 1 to 5
- rationale: short reason, maximum 18 words

Return JSON array only.
Each object must contain:
mutant_id, score, confidence, rationale
```

Pipeline. The process consists of five steps:

1) **Validate mutant pool** — checked the schema and completeness of full_sample.csv (result: 11,040 rows, status: VALID).
2) **GPT ranking** — grouped mutants into batches of 8 and called the API; each mutant received a usefulness_score from 1 to 5 along with confidence and rationale. All 11,040 mutants were scored across 1,380 API calls. Score distribution: score 1: 20 mutants, score 2: 925, score 3: 3,114, score 4: 5,573, score 5: 2,018.
3) **Random** - random selection with random_seed = 42. Seeds for each group were derived using hashlib.sha256 instead of Python's built-in hash(), because hash() is randomly salted per process, which breaks reproducibility.
4) **Selection under budget** — both branches select the top 10% of mutants in each group (project, version, class_name). Selection by group instead of from the entire pool is compulsory: since the score has only 5 levels, this leads to many ties - the overall selection resulted in 3 out of 37 groups where no GPT-selected mutants were chosen,

causing the budgets for the two branches to mismatch. Each branch selects 1,119 mutants (10.14% of the pool); the 10% budget allocation is consistent with the operating point that AL-PMT shows is sufficient to achieve near-maximum performance.

5) **Metric computation** — calculate the mutation score for each branch within each group, pair them up, and run the tests (Section C, D).

Handling edge cases: Each response is checked to ensure it contains all the required mutant_ids; if the model skips any ID, the batch is marked and not allowed to checkpoint. The entire experiment had 3/10,053 faulty batches (0.03%), all three cases due to the model returning a missing mutant_id. The pipeline records a checkpoint after each successful batch, and when rerun, it only processes mutants that have not yet been checkpointed, so the three faulty batches were rechecked in the next run. Final result: 50,694/50,694 mutants (100%) had valid points, with none of the mutants eliminated. The actual API cost, measured from tokens, was 3.57 USD for the entire experiment (14,314,218 prompt tokens, 2,368,824 completion tokens).

Deviation from the proposal: Batch sizes varied across all runs: 9,959 batches used 5 mutants/call (representing 98.1% of mutants), 90 batches used 10, and 3 batches used 25. Because mutants within the same batch appear in the same prompt, batch size can affect the assigned scores; this effect is discussed in Section VI.

Replication package: Our replication package, including datasets, prompts, scripts, and raw results, is publicly available at: https://github.com/KhoiPhanDinh/RT-SWT-003-nhom3.

C. Metrics

Mutation Score (MS) is the main metric: the proportion of mutants in the selected mutant set that are detected by the test suite: $MS = Killed/Selected$ [16] - the rate at which selected mutants are killed by the test suite. We use two deliberate levels: (a) per-group (each group has a value, used for paired comparison - the pre-registered unit); (b) overall/pooled (merging all selected mutants - reflecting the overall final user experience). Section IV will show that these two levels do not tell the same story, and that is itself a finding.

The practical effectiveness threshold: effect size rank-biserial r_rb with a success threshold of ≥ 0.30 (medium according to Cohen [25]). This threshold is not arbitrary: Jimenez et al. showed that a small effect of $A_{12} \approx 0.57$ is not practical enough for mutant selection [16], so a successful claim must exceed that threshold.

Tools: pitest 1.15.8 [23] for mutation; scipy 1.18 for Wilcoxon; pandas for data processing; all versions are recorded in requirements.txt.

D. Statistical Analysis Plan

Pre-registered test: Wilcoxon one-sided signed-rank test (H1: $MS\_GPT > MS\_random$), paired by (project, version, class), $\alpha = 0.05$. Reason: the data are paired according to the design, the MS distribution cannot be assumed to be normal, and this is the standard recommendation for evaluating algorithms with a random element in SE according to Arcuri and Briand [26]; it also reuses the design of Jimenez et al. [16] to make the results comparable.

Effect size: Matched-pairs rank-biserial r_rb; interpretation < 0.2 small / 0.2–0.5 medium / > 0.5 large [25].

Original pre-registered criteria and amendment: original criteria: $p < 0.05$ and $r\_rb \geq 0.30$ and $median(\Delta MS) > 0$. During analysis, the median criterion revealed a structural flaw: the selection size for each group k ranged from 1 to 173, making small groups' MS values roughly quantized ($k = 1 \rightarrow MS \in \{0, 1\}$, $k = 2 \rightarrow step$ 0,5) consequently, 11/37 pairs were absolute ties ($\Delta MS = 0$), and the entire sample median was mathematically pinned to 0 regardless of the direction of the effect - while the median of the non-zero pairs was +0.082. In other words, this criterion failed to measure what it was intended to measure on data with a skewed group structure. We removed the median criterion (keeping the p and r_rb thresholds) and recorded this as an approved amendment - this is a pitfall in the evaluation unit that was warned about in the PMT literature [20], and §4 reports both the original and amended results.

Subgroup analysis (pre-registered): analysis by project; if $n\_group < 10$, only descriptive statistics are reported, and no separate test is run.

IV. Results

RQ1: On Defects4J Java projects, does GPT-4o-mini-based mutant selection achieve a higher mutation score compared to random fixed-seed selection when both select the top 10% of mutants?

A. Overall mutation score

TABLE I
OVERALL MUTATION SCORE (MUTANT-WEIGHTED; SAME BUDGET OF 1,119 MUTANTS PER BRANCH).

Strategy	Selected	Killed	Mutation Score
GPT-4o mini	1,119	801	**0.7158**
Random fixed-seed (42)	1,119	750	0.6702

The absolute difference is +0.0456 (+51 additional mutants killed). Project-based selection distribution (GPT): Codec 237, CSV 64, JacksonCore 818 mutants; the random selection has an identical number by group (a characteristic of by-group budgeting).

B. Paired group-level (pre-registered unit)

According to the original pre-registered criteria ($(p < 0.05 \wedge r_{rb} \geq 0.30 \wedge median(\Delta MS) > 0)$): $p = 0.0591 > \alpha$ and $median(\Delta MS) = 0.000 \rightarrow$ fail to reject H0; specifically, r_rb=+0.350 (medium) reached the threshold. After the amendment (removing the median criterion), the conclusion remains unchanged because p is still $> \alpha$. No subgroup reached $p < 0.05$; CSV has the largest effect size (+0.571), but n=10 (Table II).

TABLE II
PAIRED BY (PROJECT, VERSION, CLASS); $N = 37$ GROUPS; ONE-SIDED WILCOXON TEST; $\alpha = 0.05$.

Subset	n	nonzero	mean MS GPT	mean MS Rnd	median GPT	median Rnd	W/L/T	p	r_{rb}
Pooled	37	26	0.784	0.725	0.808	0.714	17/9/11	0.0591	+0.350
Codec	13	8	0.875	0.829	0.923	0.923	4/4/5	0.4922	+0.028
Csv	10	6	0.760	0.682	0.809	0.733	4/2/4	0.1250	+0.571
JacksonCore	14	12	0.717	0.659	0.698	0.638	9/3/2	0.1018	+0.436

C. Tie structure and resolution

11 out of 37 pairs were absolute ties ($\Delta MS = 0$), of which 6 pairs came from groups with $k \leq 3$ mutants selected (k distribution: min=1, median=15, max=173; 2 groups had k=1). The median of the 26 non-zero pairs was +0.082. Of the 11 ties, 8 were at MS=1.0 (both branches killed cleanly - a ceiling effect), and 1 was at MS=0.

D. Robustness

(a) Mutant-weighted two-proportion z-test: difference +0.0456; z=2.34; p=0.0097 $\rightarrow$ reject H0 at the mutant level.

(b) Monte Carlo simulation (5,000 random by-group selections under the same budget): the overall MS distribution for random selection has mean=0.6700, sd=0.0126, and range [2.5%; 97.5%] = [0.6452; 0.6935]. GPT's MS (0.7158) is +3.64 sd above the mean; only 1/5,000 draws achieved $\geq$ GPT's score $\rightarrow$ experimental p=0.0002. Seed 42 (the pre-registered baseline) falls at the 52nd percentile of the distribution - a typical draw.

(c) NO_COVERAGE control: the NO_COVERAGE rate in the GPT sample was 20.7% compared to 21.8-23.5% for random selection; on the covered-only subset, MS was 0.864 (GPT) vs. 0.810 (random seed 42) - the advantage did not come from avoiding uncovered mutants.

(d) Source of the difference: the +51 additional mutant kills show strong concentration - the 3 largest JacksonCore groups (UTF8StreamJsonParser 3f/9f, ReaderBasedJsonParser 9f) contributed 32/51 = 63%; the single group with the largest difference was Codec 6f Base64InputStream (+0.75, k=4). The average percentage of groups in which GPT completely won across the Monte Carlo simulations was only 46.1%.

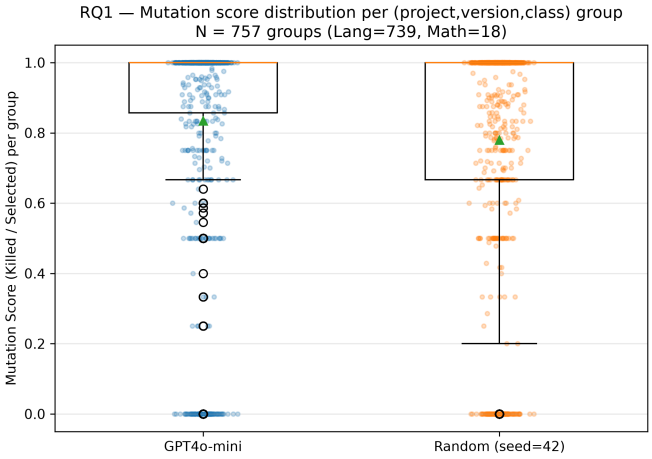

Fig. 1. Boxplot showing the MS distribution per group by strategy.

Fig. 2. Mean MS by project, with N annotation.

V. DISCUSSION

A. Explanation of key findings

The overall result reveals a two-sided picture when the data structure is examined more closely. The advantage of GPT-4o-mini is real at the aggregate level (+4.6 points, exceeding 4,999/5,000 random draws), but it is concentrated and uneven: 63% of the additional killed mutants come from the 3 largest parser groups in JacksonCore. Classes such as UTF8StreamJsonParser have dense state logic and many conditional branches - an environment where the LLM's scoring (1-5) has enough room to distinguish between shallow mutants (small constant changes with little impact) and deep mutants (those that break boundary conditions in parsing loops). Conversely, in Codec, the pool's baseline mutation score was 84.9% - a clear ceiling effect: when the test suite already killed almost all mutants, "skillful selection" and "random selection" converge to the same result, accurately reflected in the ratio of 4 wins/4 losses/5 ties and $r\_rb = +0.028 \approx 0$.

The gap between the two statistical conclusions (group-weighted p=0.059 vs. mutant-weighted p=0.0002) is a mechanical consequence of the design: group-level testing assigns equal weight to a group with k=1 and a group with k=173, and 30%

of pairs are tied (mainly small groups, coarse MS quantization, plus the ceiling effect in Codec), which negates the Wilcoxon test's ranking information. This is not a contradiction; it reflects two different questions: "Does GPT win in most classes?" (not yet proven) and "Does the mutant set selected by GPT kill more mutants overall?"

B. Comparison with prior work

These results are consistent with the earlier findings of Jimenez et al. [16]: selection based on language models does not overwhelmingly outperform random selection - they found $A_{12} \approx 0.57 - 0.58$ (small) for n-gram naturalness, while we found a pooled $r\_rb = +0.350$ for the LLM. However, there is a noteworthy qualitative difference: our effect is stratified by code type (r_rb ranging from +0.028 in Codec to +0.571 in CSV and +0.436 in JacksonCore), whereas Jimenez's results were relatively uniformly low. This suggests that a modern LLM grasps semantics well enough to make a difference in complex code - an advantage that n-gram surface statistics lack. Compared with PMT approaches [9], [10], [11], our approach is completely zero-shot (training-free, requiring no historical kill data - whereas AL-PMT still needs 10% [19]); in return, there is no calibrated probability, only a raw score from 1-5 - which is the source of the ties analyzed above. Finally, the +4.6-point MS difference from mutant selection alone is far more modest than the improvement from mutant generation using LLMs (+32.99 points in real-bug detection [4]) - which intuitively makes sense: generation opens up new space, whereas selection only optimizes within the available pool.

C. Implications

For practitioners: in a budget-constrained context, GPT-4o-mini is a safe and inexpensive choice - no worse than random selection in any project, clearly better on complex parser classes, and requiring no training or GPU infrastructure. However, for a codebase that already has a strong test suite (a high-MS architecture like Codec), investing in LLM-based selection is almost unprofitable - random selection is sufficient. For researchers: (1) the unit of analysis determines the conclusion - selection studies should report both group-weighted and mutant-weighted results and describe the structure of ties, in the same spirit as the pitfalls systematized by Delgado-Pérez et al.; (2) the raw 1-5 scale is a bottleneck - higher-resolution output (probabilities, pairwise ranking) could unlock performance currently being swallowed by ties.

VI. Threats to Validity

Internal validity: (1) The Codec ground truth failed on the first run (100% NO_COVERAGE, caused by the absence of JUnit 4 - JUnit 3 tests could not be executed by PIT): had this gone unnoticed, the entire Codec dataset would have been noise, dragging all comparisons toward zero. Mitigation: an automatic per-project guard (which fails when a project has 0 killed mutants or $> 50\%$ NO_COVERAGE) caught the error; PIT was rerun with the corrected classpath and verified on one version before running all 10; CSV and JacksonCore were confirmed to have no byte-for-byte changes; the corrupted version was saved as proof. (2) The raw GPT scores (1-5) create ties in the selection: the order in which tied mutants are selected can affect the results. Mitigation: ties are broken by mutant_id (a fixed rule, independent of the outcome); selection is done by group so that ties do not accumulate within a single project. (3) The random baseline uses a single seed: one seed could be lucky or unlucky. Mitigation: seed 42 was pre-registered in the proposal; a Monte Carlo analysis of 5,000 draws placed seed 42 at the 52nd percentile of the random distribution - a typical, unbiased representative. (4) Non-determinism of the LLM: temperature=0, a pinned model version, and per-batch checkpointing were used; however, the vendor does not guarantee absolute determinism across calls - this residual risk is noted but not fully eliminated.

External validity: The scope covers 3 Java/Defects4J projects, 20 classes, and 37 groups - the conclusion is limited to this scope and is not generalized to all Java projects, other languages, or other LLMs. Mitigation: the three projects belong to three different domains (encoding/CSV/JSON) rather than the same utility family; the report clearly states that the number of classes (not the number of projects) drives the sample size n - a lesson learned from the fact that our 3 projects have noticeably fewer groups than designs with 2 projects but many more classes.

Construct validity: (1) MS = Killed/Selected is computed while still including NO_COVERAGE mutants in the selected sample - it would be reasonable to exclude them from the denominator. Mitigation: measured - the NO_COVERAGE ratios in the two branches are close (GPT 20.7% vs. random 21.8-23.5%), and on the covered-only subset, GPT still performs better (0.864 vs. 0.810); the conclusion remains unchanged under this alternative definition. (2) The "usefulness" that the prompt asks GPT to score is not entirely consistent with the "killability" that MS measures - a mutant that is hard to kill (survives) may still be useful in that it points to a test suite weakness. We note this as a construct limitation; future studies may instead measure fault-revelation, as in [16].

Conclusion validity: (1) N=37 groups gives low statistical power for group-weighted testing; 11/37 pairs are tied, which degenerates the median criterion. Mitigation: an amendment removed the median criterion for structural reasons (approved by the instructor and documented before finalizing the conclusion); mutant-weighted and Monte Carlo results are reported in parallel; the conclusion is framed as "partial support" rather than forced into a binary outcome. (2) The CSV subgroup has n=10 - right at the pre-registered threshold (n¡10 → descriptive only); subgroup results should be read only as a reference. (3) Multiple analyses were run on the same data (risk of multiple testing). Mitigation: the pre-registered analysis is reported as-is and first; robustness analyses are clearly marked as post-hoc and have methodological justifications independent of the results.

VII. Conclusion

This research evaluates GPT-4o-mini in the role of a mutant selector - a role that has never been studied for LLMs - compared with fixed-seed selection under the same top-10% budget, on 11,040 PIT mutants [**?**] from three Defects4J projects spanning 3 different domains.

Answer to RQ1: "We investigated whether GPT-4o-mini-based mutant selection achieves a higher mutation score than same-budget random selection. Under the pre-registered group-weighted test, the evidence was insufficient to reject H0 (Wilcoxon p=0.059, r_rb=+0.350, N=37 groups); under the overall mutation score, GPT-4o-mini-based selection outperformed random selection by +4.6 points (0.716 vs. 0.670), with p=0.0097 (two-proportion z-test) and p=0.0002 (5,000-draw Monte Carlo)." The overall result is "partial support": the selection signal is real and statistically robust at the mutant level, but it is not uniform across classes - strong in complex parsers (JacksonCore, 9 wins/3 losses), and disappearing in code with near-saturated test suites (Codec, base MS 84.9%).

Key contributions: this is the first study (i) to use GPT-4o-mini as a decision maker for selecting mutants against random selection under the same budget, and (ii) to show that group-weighted versus mutant-weighted results can reverse the statistical conclusion in selection problems - a specific threat that future studies should proactively report on both sides, continuing the pitfall-systematization spirit of [**?**].

Future work: (1) Increase power where needed: expand the number of classes within the same project (running PIT at full scope instead of only on classes affected by the patch) - a direct lever for increasing the number of groups, rather than adding new projects; (2) Test the complexity hypothesis: stratify results by the cyclomatic complexity of each class to directly confirm the "LLM helps most in complex code" pattern suggested by the current data; (3) Increase score resolution: replace the 1-5 scale with pairwise ranking or calibrated kill probability - a direction already signaled by prioritization work [17] - to reduce ties, the main source of information loss in group-weighted testing.

References

[1] G. Guizzo, F. Sarro, J. Krinke, and S. R. Vergilio, "Sentinel: A hyper-heuristic for the generation of mutant reduction strategies," *IEEE Transactions on Software Engineering*, vol. 48, no. 3, pp. 803–818, 2022.

[2] J. M. Zhang, L. Zhang, D. Hao, L. Zhang, and M. Harman, "An empirical comparison of mutant selection assessment metrics," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2019.

[3] M. Ojdanic, E. Soremekun, R. Degiovanni, M. Papadakis, and Y. Le Traon, "Mutation testing in evolving systems: Studying the relevance of mutants to code evolution," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 1–39, 2023.

[4] B. Wang, M. Chen, M. Deng, Y. Lin, M. Harman, M. Papadakis, and J. M. Zhang, "A comprehensive study on large language models for mutation testing," *ACM Transactions on Software Engineering and Methodology*, 2025, to appear. [Online]. Available: https://arxiv.org/abs/2406.09843

[5] B. Wang, M. Deng, M. Chen, C. Yang, Y. Lin, M. Harman, M. Papadakis, and J. M. Zhang, "Boosting LLMs for mutation generation," arXiv preprint, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2507.08036

[6] F. Tip, J. Bell, and M. Schäfer, "LLMorpheus: Mutation testing using large language models," *IEEE Transactions on Software Engineering*, 2025. [Online]. Available: https://arxiv.org/abs/2404.09952

[7] M. Tanhaei and R. Alizadehsani, "GEM-LLM: Identifying contextual equivalent mutants via large language models; a global invariant-based approach," *Intelligent Systems with Applications*, 2025. [Online]. Available: https://github.com/tanhaei/GEM-LLM

[8] Z. Tian, H. Shu, D. Wang, X. Cao, Y. Kamei, and J. Chen, "Large language models for equivalent mutant detection: How far are we?" in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 1733–1745.

[9] J. Kim, J. Jeon, S. Hong, and S. Yoo, "Predictive mutation analysis via the natural language channel in source code," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 4, pp. 1–27, 2022.

[10] K. Jain, U. Alon, A. Groce, and C. Le Goues, "Contextual predictive mutation testing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2023.

[11] Y. Zhao, Y. Chen, Z. Sun, Q. Liang, G. Wang, and D. Hao, "Spotting code mutation for predictive mutation testing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2024.

[12] Z. Tian, J. Chen, Q. Zhu, J. Yang, and L. Zhang, "Learning to construct better mutation faults," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2022.

[13] Z. Tian, J. Chen, D. Wang, Q. Zhu, X. Fan, and L. Zhang, "LEAM++: Learning for selective mutation fault construction," *ACM Transactions on Software Engineering and Methodology*, 2025.

[14] A. Garg, M. Ojdanic, R. Degiovanni, T. T. Chekam, M. Papadakis, and Y. Le Traon, "Cerebro: Static subsuming mutant selection," *IEEE Transactions on Software Engineering*, vol. 49, no. 5, pp. 2821–2835, 2023.

[15] W. Ma, T. T. Chekam, M. Papadakis, and M. Harman, "MuDelta: Delta-oriented mutation testing at commit time," in *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 897–909.

[16] M. Jimenez, T. T. Checkam, M. Cordy, M. Papadakis, M. Kintis, Y. Le Traon, and M. Harman, "Are mutants really natural? a study on how "naturalness" helps mutant selection," in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2018, pp. 1–10.

[17] M. S. Bouafif, M. Hamdaqa, and E. Zulkoski, "PRIMG: Efficient LLM-driven test generation using mutant prioritization," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 2025. [Online]. Available: https://arxiv.org/abs/2411.08254

[18] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," *Information and Software Technology*, vol. 171, p. 107468, 2024. [Online]. Available: https://arxiv.org/abs/2308.16557

[19] M. Borsi, "Using active learning to train predictive mutation testing with minimal data," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025.

[20] P. Delgado-Pérez, S. Balderas-Díaz, and I. Medina-Bulo, "Methodological pitfalls in predictive mutation testing: Threats, impact and open challenges," *Automated Software Engineering*, vol. 33, no. 84, 2026.

[21] R. Pitts, "Random mutant selection and equivalent mutants revisited," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2022.

[22] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: A database of existing faults to enable controlled testing studies for Java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2014, pp. 437–440.

[23] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "PIT: A practical mutation testing tool for Java (demo)," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2016, pp. 449–452. [Online]. Available: https://pitest.org

[24] OpenAI, "GPT-4o mini: Advancing cost-efficient intelligence," OpenAI product documentation, 2024, model version `gpt-4o-mini-2024-07-18`. [Online]. Available: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[25] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.
[26] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*. ACM, 2011, pp. 1–10.